

Phase 2: Supabase Database

Technical documentation: persistent storage, full schema, use cases and data flows

Project	Job Agent (working name), AI job-search automation
Phase	2 of 9: Database layer
Prepared for	Naftali Murimi Njeri
Date	October 5, 2026
Builds on	Phase 0 (foundation) and Phase 1 (JWT authentication)
Leads to	Phase 3: Gmail scanner and merged OpenAI call

1. Purpose and Scope

At the end of Phase 1 the authentication system worked, but users lived in a Python dictionary inside `store.py`. Every server restart wiped it. Phase 2 replaces that dictionary with a real PostgreSQL database hosted on Supabase, so accounts survive restarts and the app has somewhere to keep everything that later phases produce.

Phase 2 has two deliverables:

- **A working users table.** The only table that Phase 2 code actually reads and writes. Signup, login and `/auth/me` now run against Supabase.
- **The complete schema.** All seven tables for the whole project are created now, as the build plan specifies, so later phases add code only and never change the database structure.

Out of scope for this phase: Gmail access, OpenAI calls, CV upload endpoints, Telegram, and the frontend. Those tables exist but stay empty until their phases.

2. What Was Built

- A Supabase project (PostgreSQL) with seven tables, indexes, constraints and Row Level Security enabled.
- `app/database.py`: one shared Supabase client for the whole app (new file).
- `app/store.py`: rewritten to query Supabase. Same three function names and return shapes as before.
- `app/config.py`: two new required settings, `supabase_url` and `supabase_service_key`.
- `.env` and `.env.example`: two new keys.
- Unchanged on purpose: `api/auth.py`, `utils/auth_utils.py`, `utils/dependencies.py`, `schemas/auth.py`.

3. Technologies and Packages

Item	What it is	Why it is used here
Supabase	Hosted PostgreSQL with an auto-generated REST API (PostgREST)	Free tier, no server to manage, kept as the database through Azure deployment to hold costs near \$0
PostgreSQL	Relational database	Foreign keys, unique constraints and check constraints enforce data rules in the database itself
supabase (Python package)	Client library for Supabase	Talks to the database over HTTPS with simple method calls; no driver or connection string setup
service_role key	Secret server-side API key	Lets the backend read and write; bypasses Row Level Security. Never leaves the server

Item	What it is	Why it is used here
Row Level Security (RLS)	PostgreSQL per-row access control	Enabled on every table with no policies, so the public anon key can access nothing
pydantic-settings	Already installed in Phase 0	Loads and validates the two new environment variables at startup

New packages: supabase only (installed with `pip install supabase`, then `pip freeze > requirements.txt`). Its dependencies, including the PostgREST client, come along automatically.

4. Architecture: Before and After

The request path from the client to the database is a straight line of small layers. Each layer has one job, which is why Phase 2 only needed to touch the data layer.

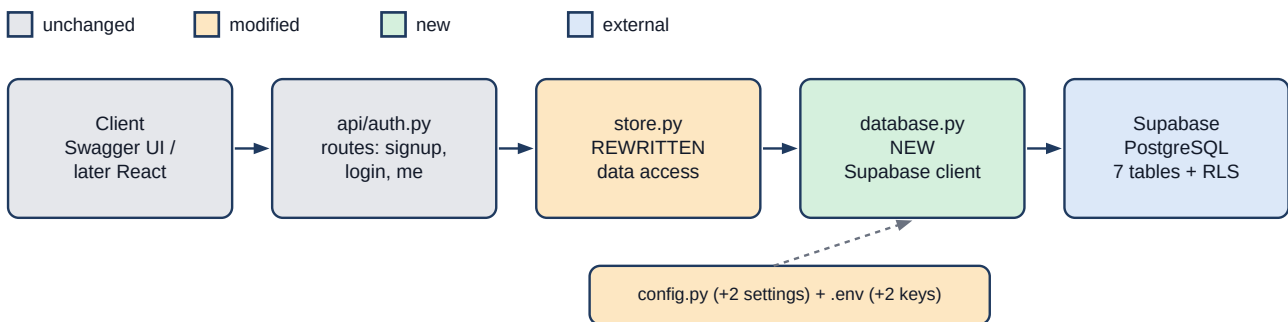


Figure 1: Layers of the backend after Phase 2. Orange = modified, green = new, grey = untouched.

Why the auth code did not change: `store.py` keeps the same contract. `create_user`, `get_user_by_email` and `get_user_by_id` take the same arguments and return the same dictionary shape, with the same keys (`id`, `email`, `hashed_password`, `full_name`, `created_at`). The routes cannot tell the difference between a Python dictionary and a database.

Design decisions

Decision	Reason
Supabase client instead of SQLAlchemy	Fewer moving parts: no ORM models, no migrations tool, no connection pooling setup. Fits the simple data access this app needs
Service key plus RLS enabled	Backend gets full access; the anon key (which is safe to expose in browsers) gets none. Two layers of protection
UUID primary keys (<code>gen_random_uuid</code>)	Not guessable or sequential, so IDs in URLs and tokens reveal nothing about user count
Emails lowercased on write and lookup	Stops "A@x.com" and "a@x.com" becoming two accounts
One shared client in <code>database.py</code>	Created once at import time and reused; avoids rebuilding a client on every request
All tables created in Phase 2	Matches the build plan; schema is stable before any feature code depends on it
ON DELETE CASCADE on <code>user_id</code>	Deleting a user removes all of their CVs, emails, settings and chats; no orphan rows
Plain <code>def</code> endpoints	The Supabase client used here is synchronous; FastAPI runs sync endpoints in a thread pool so the server stays responsive

5. Use Case Diagram

The diagram shows what the system does for its actors in Phase 2. The **User / API Client** (Swagger UI today, the React app later) triggers the three user-facing use cases. Each one includes one or more data-access use cases, which are the functions in `store.py`. **Supabase** is the external system that fulfils them.

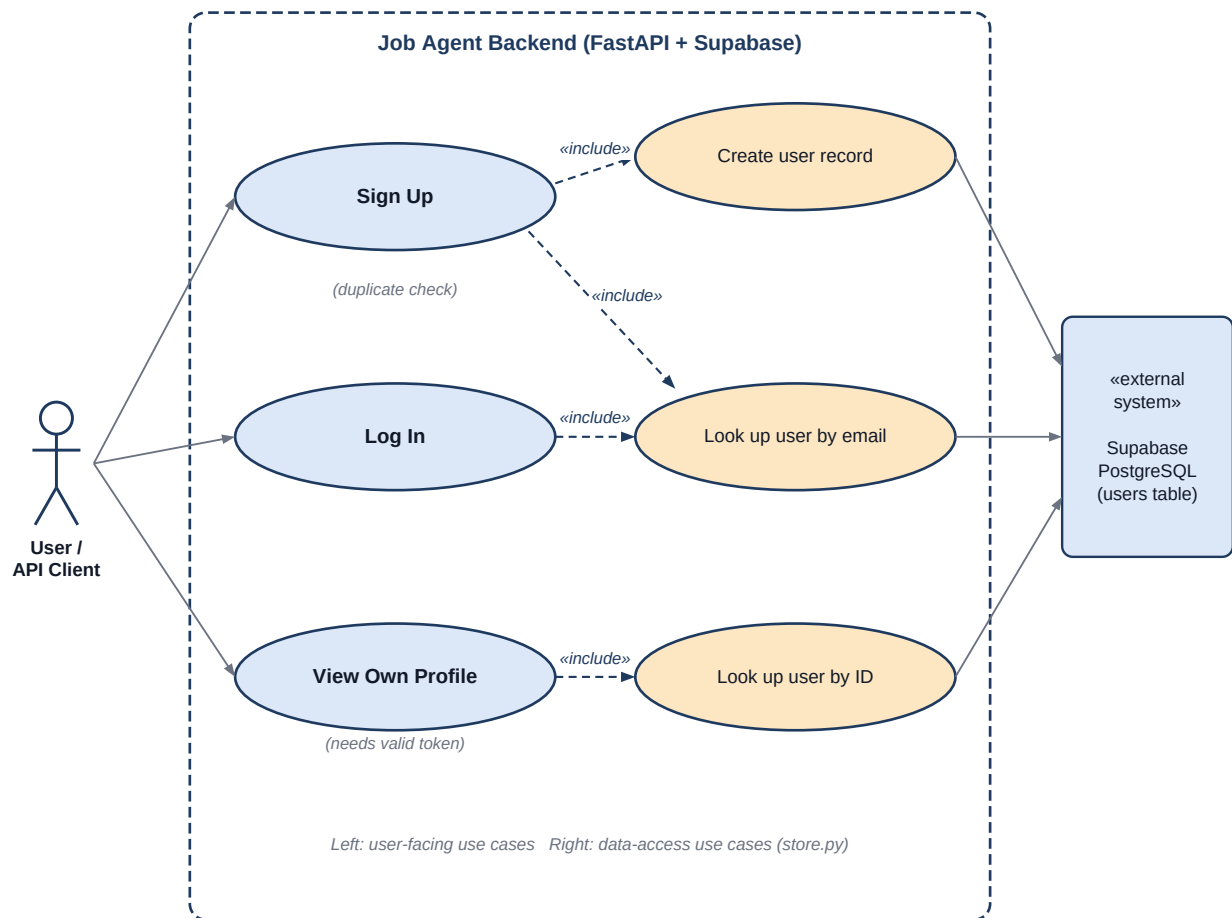


Figure 2: Use case diagram for Phase 2 (system boundary = backend).

ID	Use case	Actor	Description and outcome
UC1	Sign Up	User	Submits email, password and optional name. System rejects duplicates (400), hashes the password, stores the user, returns the user without the password. Success = 201
UC2	Log In	User	Submits email and password. System finds the user, verifies the password against the stored hash, returns a JWT valid for 24 hours. Failure = generic 401
UC3	View Own Profile	User	Sends the JWT in the Authorization header. System decodes it, loads the user by ID, returns the profile. Invalid or expired token = 401
UC4	Create user record	Supabase	Included by UC1. Inserts one row into users. Fails with a unique violation if the email already exists
UC5	Look up user by email	Supabase	Included by UC1 (duplicate check) and UC2 (find account). Returns one row or nothing
UC6	Look up user by ID	Supabase	Included by UC3. Uses the user ID stored in the token's sub field

6. Data Flow Diagrams

6.1 Level 0: Context diagram

The whole backend is one process. Data enters from the client, the backend talks to the single data store, and results return to the client.

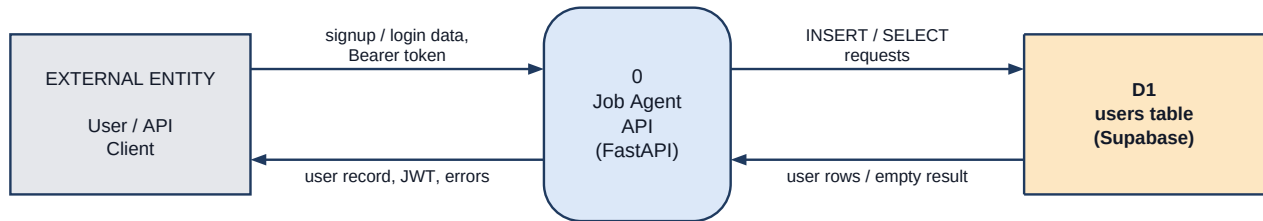


Figure 3: DFD level 0. Rectangle = external entity, rounded box = process, open box = data store.

6.2 Level 1: Processes and the users data store

Level 1 splits the backend into the three processes that touch the database. Password hashing and JWT creation or verification (Phase 1 logic) happen inside processes 1.0, 2.0 and 3.0 and need no data store of their own.

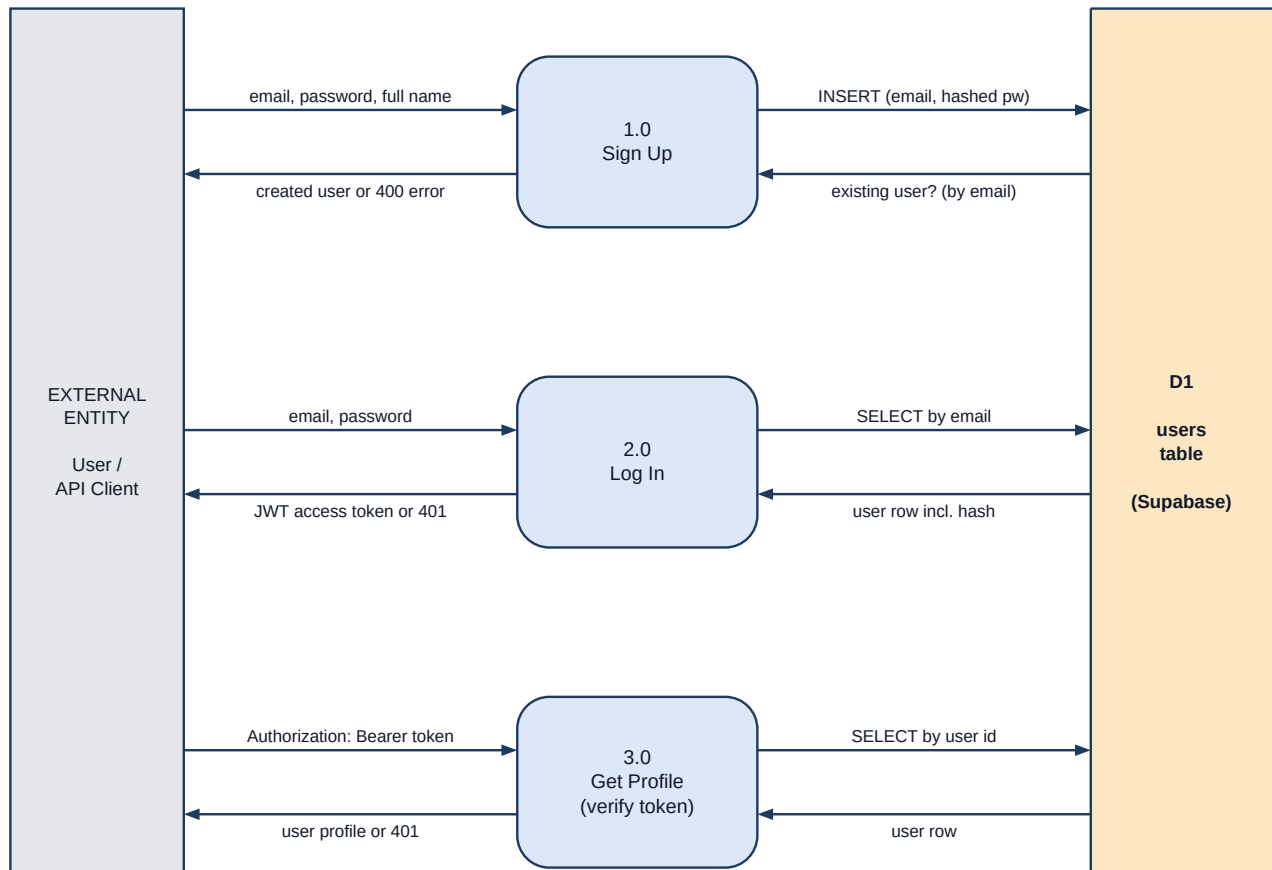


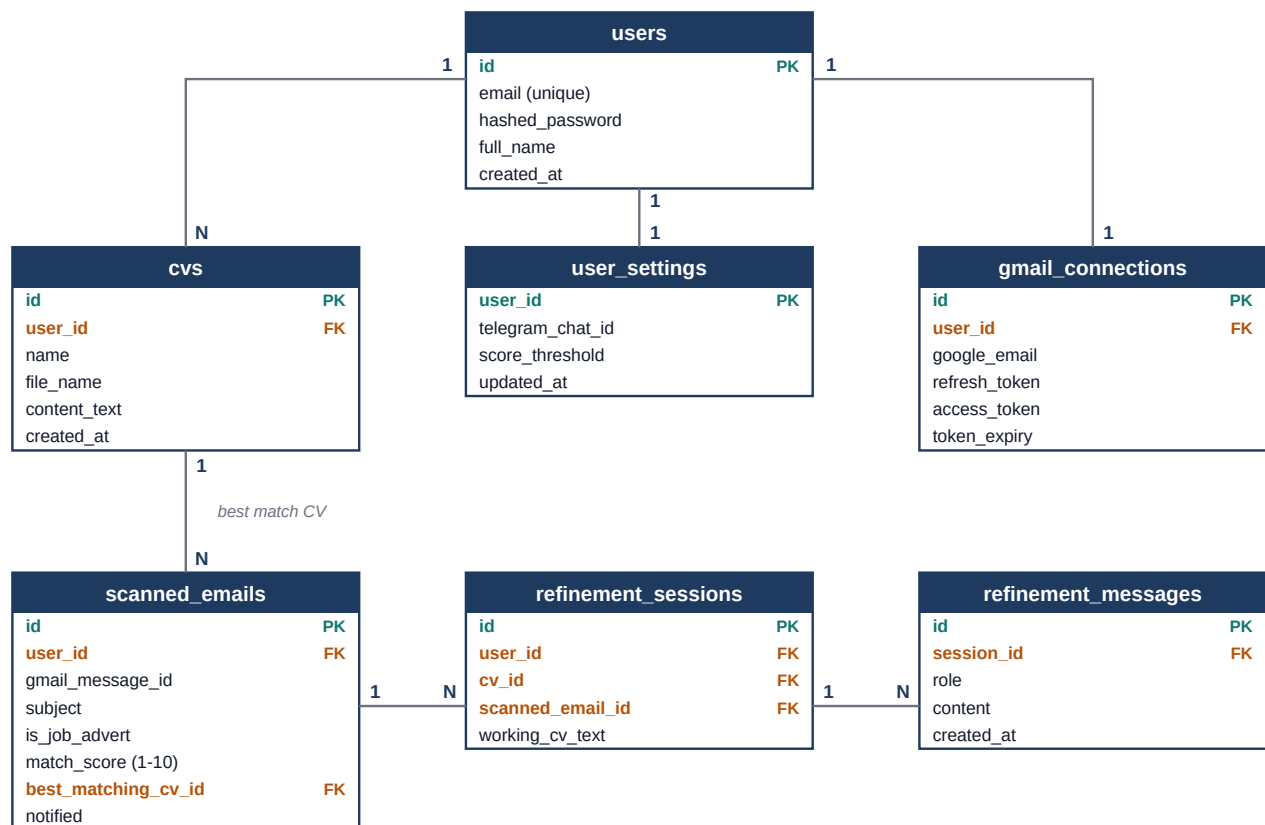
Figure 4: DFD level 1. Every arrow into or out of D1 is a call through store.py to Supabase.

6.3 Data dictionary

Data flow	Contents	Notes
Signup data	email, password, full_name (optional)	Password is plain text only in transit over HTTPS and in memory; never stored
Login data	email, password	Same handling as above
Bearer token	JWT: sub (user ID), exp (expiry), signed with SECRET_KEY	Sent in the Authorization header on protected routes
INSERT (email, hashed pw)	email (lowercased), bcrypt hash, full_name	The only write path into D1 in Phase 2
SELECT by email / by id	A single filter value	Returns zero or one row
User row	id, email, hashed_password, full_name, created_at	hashed_password is used internally but removed from API responses by the UserResponse schema
JWT access token	Signed token string plus token_type	Returned only on successful login
Errors	400 duplicate email, 401 bad credentials or token	Login error text is deliberately generic

7. Database Schema

Seven tables cover the whole project. Phase 2 code uses only `users`; the rest are created now and filled by later phases.



PK = primary key FK = foreign key Lines show main relationships (1 = one, N = many).
Every user-owned table also links to `users` through `user_id` (ON DELETE CASCADE).

Figure 5: Entity-relationship diagram (key columns shown).

Table reference

Table	Used in	Purpose and notable constraints
users	Phase 1-2	Accounts. email is unique and not null; id generated by the database
cvs	Phase 3	Multiple CVs per user. content_text holds the extracted text sent to OpenAI. Cascades on user delete
user_settings	Phase 4	One row per user: telegram_chat_id and score_threshold (default 7). user_id is both primary and foreign key
gmail_connections	Phase 6	One row per user (unique user_id): Google account email and OAuth tokens. refresh_token is sensitive and will be encrypted before use
scanned_emails	Phase 3	One row per processed email: AI classification, match_score (check 1 to 10), best_matching_cv_id, notified flag. Unique on (user_id, gmail_message_id) so no email is processed twice
refinement_sessions	Phase 7	One chat per CV and job. working_cv_text holds the evolving edited CV
refinement_messages	Phase 7	Chat history. role is checked to be user or assistant

Complete SQL (safe to re-run)

Run in Supabase: SQL Editor, New query. Every statement uses `if not exists`, so running it again changes nothing.

```
-- 1. USERS
create table if not exists users (
  id uuid primary key default gen_random_uuid(),
  email text unique not null,
  hashed_password text not null,
  full_name text,
  created_at timestamptz default now()
);

-- 2. CVS (multiple per user)
create table if not exists cvs (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references users(id) on delete cascade,
  name text not null,
  file_name text,
  content_text text not null,
  created_at timestamptz default now()
);

-- 3. USER SETTINGS
create table if not exists user_settings (
  user_id uuid primary key references users(id) on delete cascade,
  telegram_chat_id text,
  score_threshold int not null default 7,
  updated_at timestamptz default now()
);

-- 4. GMAIL CONNECTIONS
create table if not exists gmail_connections (
  id uuid primary key default gen_random_uuid(),
  user_id uuid unique not null references users(id) on delete cascade,
  google_email text,
  refresh_token text not null,
  access_token text,
  token_expiry timestamptz,
  created_at timestamptz default now()
);

-- 5. SCANNED EMAILS
create table if not exists scanned_emails (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references users(id) on delete cascade,
  gmail_message_id text not null,
  subject text,
  sender text,
  received_at timestamptz,
  is_job_advert boolean,
  category text,
  job_title text,
```

```

    company text,
    match_score int check (match_score between 1 and 10),
    best_matching_cv_id uuid references cvs(id) on delete set null,
    reasoning text,
    notified boolean not null default false,
    created_at timestamptz default now(),
    unique (user_id, gmail_message_id)
);

-- 6. REFINEMENT CHAT
create table if not exists refinement_sessions (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references users(id) on delete cascade,
    cv_id uuid not null references cvs(id) on delete cascade,
    scanned_email_id uuid not null references scanned_emails(id) on delete cascade,
    working_cv_text text,
    created_at timestamptz default now()
);

create table if not exists refinement_messages (
    id uuid primary key default gen_random_uuid(),
    session_id uuid not null references refinement_sessions(id) on delete cascade,
    role text not null check (role in ('user', 'assistant')),
    content text not null,
    created_at timestamptz default now()
);

-- INDEXES
create index if not exists idx_cvs_user on cvs(user_id);
create index if not exists idx_emails_user_score on scanned_emails(user_id, match_score desc);
create index if not exists idx_messages_session on refinement_messages(session_id, created_at);

-- ROW LEVEL SECURITY (no policies = anon key has no access)
alter table users enable row level security;
alter table cvs enable row level security;
alter table user_settings enable row level security;
alter table gmail_connections enable row level security;
alter table scanned_emails enable row level security;
alter table refinement_sessions enable row level security;
alter table refinement_messages enable row level security;

```

8. Security Design

- **Two keys, two roles.** The anon (publishable) key is designed to appear in browser code, so it must have no power over private data. The service_role (secret) key bypasses RLS and is for trusted servers only. The backend uses the service key.
- **RLS on, no policies.** With RLS enabled and no policy written, the anon key cannot read or write any table. If the anon key ever leaks, the password hashes are still unreachable.
- **Secrets stay out of Git.** The service key lives only in backend/.env, which is in .gitignore. .env.example carries blank values only.
- **Hashes, not passwords.** Only the bcrypt hash is stored. Responses never include it.
- **Never use the service key in the frontend.** The React app (Phase 5) talks to the FastAPI backend, not to Supabase directly.

Rotate the key if it ever leaks. If the service key is committed or pasted somewhere public, regenerate it in Supabase (Project Settings, API) and update .env.

9. Project Structure After Phase 2

```
Job_Agent/
|-- .gitignore
|-- README.md
|-- run_api.cmd
|-- backend/
|   |-- venv/
|   |-- .env                <- + SUPABASE_URL, SUPABASE_SERVICE_KEY
|   |-- .env.example        <- + the same two keys, blank
|   |-- requirements.txt    <- + supabase and its dependencies
|   |-- app/
|   |   |-- main.py
|   |   |-- config.py       <- MODIFIED (2 new settings)
|   |   |-- database.py     <- NEW (shared Supabase client)
|   |   |-- store.py        <- REWRITTEN (Supabase queries)
|   |   |-- api/auth.py     (unchanged)
|   |   |-- schemas/auth.py (unchanged)
|   |   |-- utils/auth_utils.py (unchanged)
|   |   |-- utils/dependencies.py (unchanged)
|   |   |-- utils/logger.py  (unchanged)
|   |-- tests/
|-- frontend/                (empty until Phase 5)
-- docs/
```

10. Key Files Explained

10.1 app/config.py (additions)

Keep every existing field. Add the two Supabase fields. They have no default, so the app refuses to start if they are missing from `.env`, which fails loudly instead of at the first database call.

```
class Settings(BaseSettings):
    # ... existing fields (app_name, secret_key, algorithm, token expiry) unchanged ...

    # Supabase (Phase 2)
    supabase_url: str
    supabase_service_key: str
```

10.2 app/database.py (new)

Creates the client once. Any module that needs the database imports supabase from here.

```
from supabase import create_client, Client

from app.config import settings

# One shared client, created once and reused everywhere
supabase: Client = create_client(settings.supabase_url, settings.supabase_service_key)
```

10.3 app/store.py (rewritten)

The same three functions as Phase 1, now backed by the database. The chain reads as: table, operation, filter, execute. `result.data` is a list of row dictionaries; an empty list means no match.

```
from app.database import supabase

def create_user(email: str, hashed_password: str, full_name: str | None = None) -> dict:
    result = (
        supabase.table("users")
        .insert({
            "email": email.lower(),
            "hashed_password": hashed_password,
            "full_name": full_name,
        })
        .execute()
    )
    return result.data[0]
```



```
def get_user_by_email(email: str) -> dict | None:
    result = (
        supabase.table("users")
        .select(".*")
        .eq("email", email.lower())
        .execute()
    )
    return result.data[0] if result.data else None

def get_user_by_id(user_id: str) -> dict | None:
    result = (
        supabase.table("users")
        .select(".*")
        .eq("id", user_id)
        .execute()
    )
    return result.data[0] if result.data else None
```

Function	Called by	SQL equivalent
create_user	signup (after hashing)	INSERT INTO users (...) VALUES (...) RETURNING *
get_user_by_email	signup (duplicate check), login	SELECT * FROM users WHERE email = ...
get_user_by_id	get_current_user (after token decode)	SELECT * FROM users WHERE id = ...

11. Setup Steps (Summary)

#	Step	Detail
1	Create the project	supabase.com, New project, name job-agent, EU region (Frankfurt), save the database password
2	Create the tables	SQL Editor, New query, paste the SQL from section 7, run. Expect "Success. No rows returned"
3	Get credentials	Project Settings, API: copy the Project URL and the service_role (or Secret) key. Not the anon key
4	Update env files	backend/.env gets real values; backend/.env.example gets blank ones
5	Install client	cd backend, activate venv, pip install supabase, pip freeze > requirements.txt
6	Edit code	Update config.py, create database.py, replace store.py
7	Run and test	run_api.cmd, then follow the checklist in section 12
8	Commit	git status (confirm .env is not listed), then git add ., git commit, git push

Environment file format. **SUPABASE_URL must be the bare project URL**, with no trailing slash, no path and no quotes:

```
SECRET_KEY=your-existing-secret-key
SUPABASE_URL=https://abcdxyz.supabase.co
SUPABASE_SERVICE_KEY=your-service-role-key
```

12. Verification Checklist

#	Test	Expected result
1	Table Editor in Supabase	All 7 tables listed, each showing RLS enabled
2	POST /auth/signup (new email)	201, user returned without password
3	Look at the users table	New row exists; hashed_password is a long scrambled string starting with \$2b\$
4	POST /auth/login	200 and an access token
5	Authorize, then GET /auth/me	Returns the same user
6	Stop the server, start it again, log in again	Login still succeeds. This is the proof the data persists (it failed in Phase 1)

#	Test	Expected result
7	POST /auth/signup with the same email	400 duplicate email
8	POST /auth/login with a wrong password	401 generic error
9	git status before committing	.env is not in the list

13. Troubleshooting

Symptom	Cause and fix
APIError PGRST125: Invalid path specified in request URL	SUPABASE_URL has extra characters. It must be exactly https://xxx.supabase.co: no trailing slash, no /rest/v1, not the dashboard link, no quotes. Fix .env, then fully restart the server. Check with: <code>python -c "from app.config import settings; print(repr(settings.supabase_url))"</code>
PGRST205: Could not find the table public.users	The SQL did not run, ran in a different project, or the schema cache is stale. Re-run the SQL, then run: <code>notify pgrst, 'reload schema';</code>
Invalid API key	Wrong or truncated key. Copy the full service_role / Secret key again. Do not use the anon or publishable key
Field required: supabase_url (on startup)	Missing or misspelled variable in backend/.env. Names must match exactly; restart the server
duplicate key value violates unique constraint (23505)	That email already exists. Use another email or delete the row in Table Editor. The API normally returns 400 before reaching this
Edits to .env not picked up	--reload watches code, not always .env. Stop with Ctrl+C and run <code>run_api.cmd</code> again
bcrypt or passlib errors after installing supabase	Re-pin: <code>pip install bcrypt==4.0.1</code> , then <code>pip freeze > requirements.txt</code>
Slow first request or connection error after days of inactivity	Free Supabase projects pause after about a week without activity. Restore the project from the dashboard and retry

14. Known Limitations and Notes

- No RLS policies are defined because only the backend accesses the database. Policies become relevant only if a frontend ever queries Supabase directly, which this design avoids.
- Gmail tokens in `gmail_connections` are stored as plain text columns for now. Encrypt them before Phase 6.
- Free-tier Supabase projects pause after roughly a week of inactivity; this matters for a personal tool that may sit idle.
- The Supabase client is synchronous. If the app later needs heavy concurrency, revisit the async client.
- Schema changes from here on should be saved as SQL files under `docs/` so the database can be rebuilt from scratch.

15. Outcome and Next Phase

When the checklist passes, Phase 2 is complete: accounts persist across restarts, the full schema is in place, and the database is locked down to the backend alone. Authentication behaves exactly as in Phase 1, which confirms that the layered design (routes, auth utilities, store, database) lets one layer change without the others noticing.

Phase 3 (Gmail scanner and merged OpenAI call) is next. It will add CV upload and storage in the `cvs` table, Google Cloud OAuth credentials for Gmail, a scan function that fetches job-alert emails, one OpenAI call that classifies each email and scores it against all of the user's CVs, and results saved to `scanned_emails`, triggered manually so it can be tested against a real inbox.